MLTorq by GearOps

Architecture, features, integrations, advantages and extensibility

mltorq.com · built on the RivetCI CI/CD engine · prepared for Sudhakar (founder) for YC, a16z Speedrun and the demo video · 30 Oct 2026

How to read this: every claim points to code in the gearops monorepo. "Built" means the code exists today. "Partial" means it works with limits that are stated. "Roadmap" means not built. Measured numbers come only from the demo jobs named in the text. Market facts in section 9 carry links.

Contents

1. Overview and core idea
2. Architecture
3. Features
4. Integrations
5. Security model
6. Advantages and competitors
7. Future extensibility
8. Measured results
9. Market trends and fit
10. Honest gaps and roadmap
11. Where everything lives
12. Sources

1. Overview and core idea

MLTorq runs every step of shipping software and AI as a typed pipeline step. A build, a security scan, a LoRA fine-tune, an eval gate and an LLM cost comparison are all the same kind of thing: a task type. A task type is a script plus typed form fields. A pipeline is an ordered chain of task types. One engine (RivetCI) runs the chain on the customer's own runners (Docker, Kubernetes or AWS ECS), streams the logs, hides secrets, and stops the chain when a gate fails. The result: CI/CD, ML training, evaluation, security and LLM cost routing share one engine, one secrets store, one log trail and one way to add new capability.

Why this is the moat.

- **New capability is data, not a new product.** Each built-in step is a script and a field list in a manifest.json, turned into database rows by a generator. Adding "LLM: Cheapest passing model" did not change the engine.
- **Every step gets the platform for free.** Secrets resolution and masking, runner limits, live logs, S3 log archive, stop and timeout, and the job page all apply to any task type.
- **Gates are just steps that exit non-zero.** The same mechanism blocks a deploy for a CVE, a regressed prompt, or a fine-tune that did not beat the base model.
- **Cost is a log line.** Any step can print GEAROPS_METRIC {json}; the orchestrator stores it, prices it from an editable table, and shows it on the job page.

2. Architecture

2.1 Components (single-VM install)

Source: deploy/docker-compose.vm.yml, deploy/up.sh, deploy/caddy/Caddyfile.vm.

Component	What it is	Role
caddy	Caddy 2 reverse proxy	Only service with open ports (443, plus 80 for the Let's Encrypt challenge and redirect). Sends /api/* to the orchestrator and everything else to the UI. Sets HSTS and nosniff headers.
orchestrator	Spring Boot (Java), "the brain"	REST API, auth (JWT), pipelines, task types, secrets, runners, jobs, cost and routing services, AI pipeline builder. Seeds the "Docker (this server)" runner on first start.
spinner	Spring Boot (Java), "the hands"	Consumes job messages and runs each task on the chosen runner: Docker container, Kubernetes pod or ECS Fargate task. Decrypts secrets, masks logs, enforces limits.
mysql	MySQL 8.0 (database gearml)	All state: users, roles, pipelines, task_type and task_type_field, params, jobs, secrets (encrypted), llm_price, job_step_metric, GPU offers, cost plans.
kafka	Apache Kafka 3.9 (KRaft, one node)	Job and log messages between orchestrator and spinner. Internal network only. An embedded Artemis profile exists for smaller installs.
ui	React app (mycadence/)	Pipeline canvas, task forms, job page with live logs and the "Model accuracy vs cost" card, Runtime, Secrets, MLGear pages.
keystore-init	One-shot container	Creates the AES-256 keystore used for ENCv1 field encryption if missing. Shared read-only by orchestrator and spinner.

Secrets for the stack itself (database, JWT, keystore password, optional provider keys) are read at start by deploy/up.sh from AWS Secrets Manager and SSM into a mode-600 file on tmpfs. No secrets live in the compose file or in git.

2.2 The flow of one job

1. A user builds a pipeline on the canvas (or with the AI builder) and saves it. Each step's answers become param rows; secret fields are stored encrypted or as ${secret:NAME} references.
2. The user starts a job and picks a runner. JobService copies steps into job tasks. Blank answers get field defaults.
3. The orchestrator resolves ${secret:NAME} (user scope first, then global) and checks built-in steps for unsafe values (BuiltinParamGuard: no newlines or shell-special characters).
4. The job message (secrets still ENCv1-encrypted) goes over Kafka to the spinner, with the runner section (size, timeout, kubeconfig or ECS settings).
5. The spinner decrypts secrets, registers them with the log masker, writes a 0600 env file the task sources and deletes, and runs the script on the runner.
6. Each log line is masked, published, masked again by the orchestrator (FileLogBuffer) and streamed to the browser over SSE.
7. Lines starting GEAROPS_METRIC are stored in job_step_metric. A step that exits non-zero fails the job and later steps (such as deploy) do not run.
8. When a task ends its log moves to S3 (logs/ prefix, instance role, no keys). The job page reads it back from there.

2.3 Runners (where the pipeline runs)

Source: RunnerService.java, RunnerSpec.java, spinner DockerPipelineRunner / KubePod / EcsTaskRunner. Every job must name its runner; there is no silent fallback.

Runner	How it works	Status
Docker (this server)	Jobs run as sibling containers on the VM. Default size 1.5 CPU, 2 GB, no swap, 512 processes, 30-minute timeout, one job at a time on the 8 GB demo VM. Admin-managed, shared by all users.	Built (used by all demo pipelines)
Kubernetes	Any cluster via kubeconfig (EKS, AKS, GKE, minikube). Namespace is always the owner's own (gearops-u<id>). Sizes capped server-side. Optional spot node selector and toleration.	Built; Enterprise packaging
AWS ECS Fargate	Region, cluster, subnets, security groups validated on save. Auth by assumed role with a server-generated external ID (preferred) or encrypted keys. Logs via CloudWatch routed back into the job log.	Built; CPU only (Fargate has no GPUs)
GPU pool	A GPU pool links to a Kubernetes runner and sets GPU count and type for the pod.	Built for Kubernetes; ECS GPU spot is a design note only

Server-side caps (defaults): up to 8 CPU, 16 GB, 4 GPUs, 6-hour job timeout per runner.

2.4 Task-type model

Source: docs/learning/task-types.md, TaskType / TaskTypeField entities, orchestrator/src/main/resources/scripts/*/manifest.json.

Object	Meaning	Stored in
Task type	The recipe: name, category, script (command with ${field} placeholders), optional second script, built-in lock, hidden-from-AI flag, optional deploy target type.	task_type
Field	One form question: name, type (text, textarea, select), label, help, default, required, options, is_encrypted.	task_type_field
Pipeline step	A task type used once in a pipeline, with answers.	pipeline_task_type + param
Job / run	One execution of a pipeline on one runner.	job, job_run, task_run

Built-in task types (is_builtin = 1) are locked: PUT/DELETE answer 403 and the entity refuses updates. Their scripts change only through the .py/.sh file plus the matching scripts/gen_*.py generator, which validates the script and writes an idempotent SQL seed.

2.5 Environments and deploy targets

Concept	Question it answers	Examples
Runtime / runner	Where does the pipeline run?	Docker, Kubernetes, ECS, GPU pool
Environment	What toolchain does the job need?	OS + language + build tool (Env Setup task type 1). No default: each pipeline picks one.
Deploy target	Where does the app land?	15 built-in types: EC2, Lambda, ECS, EKS, S3 static site (AWS); Azure VM, Functions, AKS, Container Apps; GCP Compute Engine, Cloud Run, GKE; SSH host, Docker host, Kubernetes (on-prem).

Saved targets keep their secrets (SSH keys, kubeconfig, cloud keys) encrypted; the API returns them redacted. At run time a step that names a saved target gets its fields, and login secrets reach the job as SECRET_<field>. App secrets for the deployed app (appSecrets) are pushed the native way per target (Kubernetes Secret, ECS task definition, Lambda environment, and so on).

2.6 Eval gates

Gate step	What it checks	Blocks when
Eval Gate	Runs your app's eval command on a dataset, scores offline (no LLM key), compares with a baseline.	Score drops more than the allowed tolerance.
LLM Eval	Scores base model and base + LoRA adapter on test prompts (exact, contains or regex match).	Fine-tuned model does not beat base by the margin, or is below the minimum score.
Security scans	Gitleaks, dependency scan, Semgrep, Trivy.	Findings at or above the threshold (e.g. HIGH), unless fail-on-findings is off.
Titanic promotion gate (MLGear)	Compares a new model's metric with production.	New model is not better; production is kept.
LLM: Cheapest passing model	Candidate LLMs on a labelled eval set.	No candidate reaches the threshold.

2.7 Model routing and cost tracking

1. The "LLM: Cheapest passing model" step (llm_routing.py) runs each candidate model on the same labelled eval file, cheapest first. Mode "first" stops at the first that passes; mode "compare" scores all.
2. For each model it prints GEAROPS_METRIC {model, score, tokensIn, tokensOut, misses…} and one result line.
3. ModelRoutingService sees the prefix and saves the JSON in job_step_metric.
4. GET /orchestrate/jobs/{id}/savings prices tokens from the editable llm_price table (list prices, labelled estimates) and returns cost per model, cost per 1,000 requests, the cheapest passing model and the saving versus the baseline.
5. ModelSavingsCard on the job page draws "Model accuracy vs cost" with a star on the winner. Prices live in the orchestrator, so a price change needs no re-run.

Safety nets in the step: per-call timeout, two retries on 429/5xx, a hard call cap, max output tokens, and a Mock provider for free rehearsal.

2.8 GPU offers and cost plans (MLGear)

- **GPU price catalog.** gpu_offer rows from five live price adapters (AWS, Lambda, RunPod, TensorDock, Vast.ai), refreshed on a schedule, plus manual rows. Admins pick which price is used for estimates and metering. Failures keep the last known prices.
- **Cost plans: Plan, Guard, Learn.** Predict a cost range from a rate card, set a budget and an on-exceed action, meter live. CostGuardSweeper checks every 15 s and stops the bound job when the budget is passed. Learn stores actual vs predicted.
- **Self-stop in the fine-tune step.** LLM Fine-tune has budgetUsd and hourlyRateUsd fields, checkpoints to S3 and resumes on retry or spot interruption.

Limits: predictions are heuristics (shown as estimates), pause equals stop (no checkpoint resume in the runners), spot price is an estimate (0.35 x on-demand).

2.9 Roles

Role	Can	Cannot
Admin (ROLE_ADMIN)	Everything, plus global secrets, the shared Docker runner, LLM price and GPU offer edits, cost rate card.	-
Developer (default at sign-up)	Build and run pipelines, own secrets, own runners (in own namespace), pick the shared Docker runner.	Manage global secrets, shared runners or price tables; delete or change admin settings.

3. Features

Area	Feature	Status
Pipelines	Visual canvas; ordered steps; starter templates (Spring Boot CI, Titanic MLOps, LLM fine-tune + eval gate)	Built
Pipelines	AI pipeline builder and AI task-type generator (Anthropic, structured output, server-side cleanup, one repair round)	Built
Task types	Typed steps for Env Setup, Git clone, builds (Maven, Gradle, npm, Python, .NET), tests (JUnit, Pytest, Mocha), Docker build, 15 deploy targets, 4 scanners, ML steps	Built
Task types	Script checker: deny-list, ShellCheck if present, optional OPA/Conftest, placeholder validator	Built
ML	LLM Fine-tune (LoRA, transformers + peft): smoke mode on CPU, gpu mode, S3 checkpoints, spot resume, budget stop	Built
ML	Download model weights (Hugging Face, optional S3 cache)	Built
ML	LLM Eval and Eval Gate	Built
ML	Batch embeddings (open sentence-transformers / BGE models, CPU, Parquet to S3)	Built
ML	Titanic MLOps loop: load, train, evaluate, register, promotion gate, deploy model API, PSI drift check that starts a retrain	Built (deploy step needs a Kubernetes runner)
LLM cost	Cheapest passing model, compare mode, savings endpoint, editable LLM price table, job-page card	Built
GPU cost	GPU offer catalog with live adapters, cost plans with budget stop	Built; predictions heuristic
Security	4 scanners as gates; secrets page with ${secret:NAME}; log masking	Built
Runtime	Docker, Kubernetes, ECS Fargate runners; GPU pools; spot capacity on Kubernetes	Built
Logs	Live SSE log tail; finished logs in S3	Built
Editions	package.ps1 builds RivetCI Community, RivetCI EE, MLGear EE	Built

4. Integrations

Only what the code supports. "Partial" is explained in the last column.

Integration	What it is used for	Status
GitHub	OAuth sign-in; GitHub App install with installation tokens to list repos; GitHub Clone and Public Git Clone task types	Built. No push/PR webhook triggers yet
AWS S3	Task log archive (instance role); fine-tune checkpoint sync; weights cache; embeddings input/output; S3 static-site deploy	Built
AWS ECR	Hosts the GearOps images the VM pulls (up.sh logs in with the instance role)	Partial: platform hosting, not a pipeline registry step
AWS ECS	Fargate runner and "Deploy to ECS" target	Built (CPU runner)
AWS EKS	Kubernetes runner via kubeconfig; "Deploy to EKS" target; spot label documented	Built
AWS Secrets Manager / SSM	Stack secrets and config at start (up.sh)	Built for the platform; not yet a backend for user secrets
Kubernetes (any)	Runner, GPU pools, on-prem deploy target; AKS, GKE targets	Built
Docker	Default runner; Docker Build step; Docker host deploy target	Built
Azure, GCP	Deploy targets (VM, Functions, AKS, Container Apps, Compute Engine, Cloud Run, GKE)	Built as scripts; not all verified end to end
Anthropic	AI pipeline/task-type builder; routing step provider	Built

OpenAI-compatible APIs	Routing step via base URL: OpenAI, vLLM, Ollama, LiteLLM and similar	Built (generic chat completions)
Hugging Face	Model weights download, gated models via encrypted HF token, LoRA base models, embedding models	Built
Gitleaks, Semgrep, Trivy, OWASP Dependency-Check, npm audit, dotnet list package --vulnerable	Security scan task types (pinned tools downloaded at run time)	Built; needs internet on first run
GPU clouds	Price adapters: AWS, Lambda, RunPod, TensorDock, Vast.ai	Partial: prices only, no provisioning
Slack	Notify task type (older type, webhook URL as secret)	Partial: legacy step, not re-verified

5. Security model

Control	How it is enforced	Where
Encryption of secrets	AES-GCM ENCv1 tokens with a 256-bit key from a keystore created per install. Secret fields, saved targets, presets, LLM keys and GPU billing credentials are encrypted at rest. Values are write-only in the API.	AesGcmService, SecretService, keystore-init
Secret references	${secret:NAME}: user scope first, then global. Unknown names fail the job without printing values. Global secrets are admin-only.	SecretRuntimeService, SecretController
Delivery to the job	Decrypted only in the spinner, written to a 0600 env file the task sources and deletes; built-in scripts read SECRET_<field>.	TaskSecretEnv, CommandResolver
Masking	Spinner masks each value (plus base64 and URL-encoded forms, and any ENCv1 token) before publishing; orchestrator masks again. Scripts and exec commands are no longer logged.	SecretMasker (both services), FileLogBuffer
Untrusted pull requests	There is no PR- or fork-triggered run path today: jobs start only from a signed-in user. So untrusted PR code never receives secrets. This must stay true when PR triggers are added (roadmap).	By design; no webhook trigger code
Throwaway job containers	Per job: 1.5 CPU, 2 GB, no swap, 512 processes, 30-minute timeout, no host mounts, no Docker socket, no spinner env, no-new-privileges, removed when done.	DockerJobLimits, docker-compose.vm.yml
Network isolation	Jobs join their own bridge network (gearops-jobs, inter-container traffic off), never the app network. The cloud metadata IP 169.254.169.254 is dropped by iptables, so jobs cannot use the VM role.	deploy/up.sh
Kubernetes isolation	Namespace forced to the owner's own; kubeconfig checked by KubeconfigPolicy; private API endpoints refused by default.	RunnerService, KubeconfigPolicy
Built-in step safety	Built-in scripts are locked; their inputs may not hold newlines or shell-special characters. New scripts pass the script checker.	BuiltinParamGuard, ScriptCheckerService
Roles	Admin vs developer limits (section 2.9). Owner checks on runners, targets and secrets.	AdminCheck, controllers
Edge	HTTPS only via Caddy (Let's Encrypt), HSTS; MySQL and Kafka not exposed.	Caddyfile.vm, compose
Customer data	The product is self-hosted: database, logs (customer S3 bucket), models and secrets stay in the customer's account. GearOps runs no shared multi-tenant service. The mltorq.com demo VM is GearOps's own and holds only demo data.	Deployment model

Known limits: the spinner holds the Docker socket on the single-VM install (root-equivalent on that VM; job containers never get it). On Kubernetes the env-file content still travels inside the exec request; a per-job Kubernetes Secret is planned. GEAROPS_METRIC lines are not signed: they are what the step reported.

6. Advantages and competitors

Advantage	Concrete mechanism in the code	Evidence
One engine for the whole AI lifecycle	Fine-tune, eval, scan, LLM routing and deploy are all task types on the same runner, secrets and log path.	Demo pipelines 391-398 on one Docker runner
Runs in your own infrastructure	Self-hosted compose stack; runners in your Docker, Kubernetes or AWS account; logs to your S3.	docker-compose.vm.yml, RunnerService
Cost on every run	GEAROPS_METRIC lines, job_step_metric, llm_price, savings endpoint, GPU offers, cost plans with budget stop.	Job 2788: Haiku 96% at $0.009 per 1,000 tickets
Gates that block bad models	Any step that exits non-zero stops the chain before deploy; Eval Gate and LLM Eval compare against baseline or base model.	392: noisy fine-tune blocked; 393: vulnerable sample blocked
Nothing hidden	Every built-in script is a readable file in the repo (Python bodies inlined line by line); custom task types are the user's own scripts.	orchestrator/src/main/resources/scripts/

Model and cloud neutral	Anthropic, any OpenAI-compatible endpoint (including self-hosted vLLM/Ollama), Hugging Face weights; Docker, any Kubernetes, ECS; 15 deploy targets across AWS, Azure, GCP and on-prem.	llm_routing.py, deploy/manifest.json

Brief competitor contrast

General product knowledge, not re-verified this session. Check current feature lists before quoting.

Product	Where they are stronger today	Where MLTorq differs
Harness	Mature enterprise CI/CD, governance, SSO/RBAC/audit, certifications, large integration catalog, sales team.	ML steps (fine-tune, eval gates, LLM cost routing) are first-class task types in the same engine.
ZenML	Python-native ML pipelines, many stack integrations, artifact and lineage tracking, active open-source community.	Also does CI/CD, security scans and deploy targets; gates and cost on the same job page.
ClearML	Experiment tracking, data versioning, GPU agent scheduling and serving.	Pipeline-first with CI/CD and security gates; LLM cost comparison per run.
LiteLLM	Live request gateway across many providers with budgets and per-request routing and spend tracking.	Chooses the model before deploy with a labelled eval and a pass mark; not in the request path.
SageMaker, Vertex AI, Azure ML	Managed GPUs, registries, hosted endpoints, compliance, scale.	Runs across clouds and on-prem in the customer's own account; scripts visible and editable.

7. Future extensibility

Most new capability plugs in as data or as one small class, because the engine only knows "run this script with these typed values on this runner". The last column says whether core engine code must change.

Extension	How it plugs in today (real extension point)	Core change?	Status
New task type	Write a script and a manifest.json (fields, env map, secretEnv map); a scripts/gen_*.py generator validates it and writes an idempotent SQL upsert. Or create it in the UI/API (POST /orchestrate/tasktype/create), checked by ScriptPlaceholderValidator and ScriptCheckerService, or drafted by the AI task-type generator. New types appear in the AI builder catalog unless hidden_from_ai = 1.	No	Built. No general edit/delete API for task types yet
New metric or cost signal	The step prints GEAROPS_METRIC {json}. ModelRoutingService stores any such line in job_step_metric.	No to store; a new chart needs UI work	Built (storage); card is routing-specific
New LLM provider	Any OpenAI-compatible endpoint works by setting Base URL (OpenAI, vLLM, Ollama, LiteLLM). A provider with its own API is a new branch in llm_routing.py, a task script. Prices are rows in llm_price (admin tab).	No	Built: anthropic, openai-compatible, mock
New scoring: custom check	Eval Gate runs any eval command you supply (regex, JSON schema, unit test, your own script) and gates on its score. LLM Eval supports exact, contains and regex.	No	Built
New scoring: judge model	Planned as a "scorer" select field (exact label / judge / custom) plus judge model and rubric fields on the routing step, with judge cost reported separately. Scoring is the only part that changes; threshold, cost per 1,000 and the card stay.	No (task script and fields)	Roadmap
New deploy target type	Add an entry to scripts/deploy/manifest.json and a <typeKey>.sh; gen_deploy_task_types.py validates and regenerates task type, fields and the target form. Users cannot add target types themselves (fixed set, API answers 400).	No Java change; ships in a release	Built (15 types)
New runner kind	RunnerSpec kinds (docker, k8s, ecs) are validated in RunnerService and executed by a spinner class (DockerPipelineRunner, KubePod, EcsTaskRunner) behind the Executor interface. Any Kubernetes cluster needs no change.	Yes, contained (one validator + one executor)	Built: 3 kinds. Micro-VM runner: roadmap
New GPU price source	Implement GpuPriceAdapter (provider, sourceUrl, missingConfig, fetch). Spring picks it up; the refresh job calls it.	One class	Built: 5 adapters
New log store	Implement LogStore (today LocalFileLogStore and S3LogStore).	One class	Built: local, S3
Messaging	MessagePublisher / LogPublisher: Kafka by default, embedded Artemis by profile.	No (profile)	Built
Secret backends	Pipelines reference ${secret:NAME}, so the store can change without touching pipelines. Vault and AWS Secrets Manager backends are designed.	Yes, in SecretService only	Roadmap
Triggers	Today: start from UI/API, seed script, MLGear drift check starts a retrain, cost guard stops jobs. Git push/PR webhooks and schedules would create jobs through the same JobService path.	Yes, new trigger endpoint	Roadmap (push/PR, cron)
Integrations (tools, APIs)	Any CLI or API a script can call becomes a task type; secrets via encrypted fields.	No	Built pattern
Pipeline templates	Rows in pipeline_template / pipeline_template_step from a generator script.	No	Built

8. Measured results

Only these numbers are cleared for use. Re-run before recording; scores can shift a little between runs.

Job / pipelines	What ran	Result
Job 2788	Support-ticket classification, three Claude models, compare mode	Haiku 96% at $0.009 per 1,000 tickets; Sonnet 96%; Opus 94%. Haiku chosen as cheapest passing.
391 / 392	LoRA fine-tune on CPU, good vs noisy data, then LLM Eval gate	About 100 s each. 391 passes; 392 is blocked at the gate.
393 / 394	Gitleaks, dependency scan, Semgrep, Trivy, then deploy echo	393 (vulnerable sample) blocked; 394 (clean sample) passes.
395-398	Support ticket triage: 395 mock rehearsal; 396-398 real calls	Cheapest passing model chosen (Haiku). Dollar cost not stored for 396-398.

9. Market trends and how MLTorq fits

Each trend links to a source opened or found on 30 Oct 2026. "Opened" means the page was read in full; "excerpt" means only the search excerpt was read, so treat it as unverified. No market-size figures are used.

Trend (source)	MLTorq mechanism that fits	Where the trend works against MLTorq
Small, task-specific models. Gartner predicts that by 2027 organizations will use small task-specific models at least three times more than general LLMs [1, opened]. NVIDIA researchers argue small models suit most agent calls [4, excerpt].	LLM Fine-tune (LoRA) runs on CPU in smoke mode and on GPU runners for real runs; LLM Eval only lets the adapter through if it beats the base model. 391/392 show the gate on a 135M model in about 100 s.	Real fine-tunes need curated data and GPUs. MLTorq has no data labeling or curation. The demo test set re-asks training questions, so it proves the gate, not generalisation.
Prices for a fixed level of quality fall fast: about 47% per quarter since 2023 per Epoch AI, with the caveat that real users rarely switch models [2, opened].	"Cheapest passing model" re-runs the same eval when new models ship; prices live in an editable table, so the savings view updates without re-running.	As prices fall, the dollar saving per run shrinks (job 2788: $0.009 per 1,000 tickets). The pitch must lean on gates, governance and one engine, not cost alone.
Open-weight models trail the frontier by about four months on Epoch's capability index [3, opened].	OpenAI-compatible provider reaches self-hosted vLLM or Ollama; Download model weights pulls from Hugging Face; jobs run in the customer's own runners; GPU offer catalog compares rental prices.	Open weights hold a small share of enterprise AI spend and in-house open-weight builds report weak ROI [6, excerpt]. MLTorq does not serve models in production (only the Titanic demo API).
Composite, multi-model systems: Gartner advises composite approaches with multiple models and steps [1, opened].	Pipelines chain steps; routing, eval and fine-tune are separate steps that can be combined per use case.	Agent workloads route per request at run time; MLTorq picks a model per task before deploy and is not in the request path.
Dynamic routers often fail to beat a random pick between two well-chosen models; the roster drives accuracy [5, opened].	MLTorq picks the roster offline with your labelled data and a pass mark, which is the part this paper says matters most.	Vendors report large savings from in-harness routing (e.g. Factory, 58% [7, excerpt, vendor claim]). The result is only as good as the eval file (50 tickets in the demo).
Regulation asks for documented evaluation: EU AI Act enforcement powers for general-purpose model obligations start 2 Aug 2026 [8, excerpt; verify dates].	Gate reports (JSON), job history and S3 logs give a record of what was tested before deploy.	No audit log, SSO or SOC 2 yet; MLTorq is not compliance software and should not be pitched as such.

Where the edge is: the market is moving to many smaller, cheaper, often open models chosen per task. That makes "test, gate, and price every model change inside the same pipeline that ships the code" more valuable over time, even as the per-call price falls. The risk is that this layer gets bundled into cloud platforms, CI vendors or gateways.

10. Honest gaps and roadmap

Item	Today	Plan
Runner agent	The spinner needs a Docker socket, a kubeconfig or an AWS role from the GearOps host.	Roadmap: an agent inside the customer network that pulls work.
SSO, fine-grained RBAC, audit log	Two roles (admin, developer) and owner checks.	Roadmap (enterprise bundle).
SOC 2	Not started.	Roadmap.
Micro-VM isolation	Hardened containers; Docker socket on the single-VM install.	Later: micro-VM runner.

Git triggers	No push/PR webhooks or schedules.	Roadmap; keep secrets away from untrusted PRs.
Judge-model scoring	Exact label, contains, regex, or your own eval command.	Roadmap (section 7).
Secret backends	Built-in encrypted store only.	Vault, AWS Secrets Manager.
Cost plans	Heuristic predictions; pause = stop; spot price estimated; no auto-calibration.	Repo-aware predict, learn loop, real spot prices.
Demo rough edges	Step 1 uses embedded demo files, not a Git clone; dollar cost not stored on 396-398; AI builder cannot pick a saved deploy target.	Switch to Public Git Clone; fix cost storage; target picker.
Housekeeping	Old names in code (Cicada, DevXP, Anvil CI); some older task types lack descriptions; credentials once committed to git history must be rotated before the repo is public.	Pre-launch checklist in ROADMAP.md.

11. Where everything lives

What	Repo path (gearops monorepo)
Orchestrator (API, services)	orchestrator/src/main/java/com/cicada/orchestrator/...
Built-in task scripts + manifests	orchestrator/src/main/resources/scripts/<area>/ (deploy, security, eval_gate, llm_eval, llm_routing, lora_finetune, model_weights, batch_embeddings, mlgear, demo_files, build_tasks)
Generators (scripts to SQL)	scripts/gen_*.py; scripts/readable_py.py
Routing and cost	ModelRoutingService, ModelRoutingController, CostConsciousnessService, CostGuardSweeper, mlgear/gpu/*
Secrets	SecretService, SecretRuntimeService, SecretMasker, AesGcmService; spinner TaskSecretEnv, CommandResolver
Runners	RunnerService, RunnerSpec; spinner DockerPipelineRunner, DockerJobLimits, KubePod, EcsTaskRunner, EcsLogRouter
Spinner (executor)	spinner/src/main/java/...
UI	mycadence/src (pages, features, components)
Deploy (VM)	deploy/docker-compose.vm.yml, deploy/up.sh, deploy/caddy/Caddyfile.vm, deploy/systemd/gearops.service
Laptop helper scripts	deploy/laptop/*.ps1 (S3 logs, demo seed, demo user)
AWS / Kubernetes setup	scripts/aws/setup_ecs_runner.*, scripts/k8s/rivetci-runner-access.yaml
Sample data	examples/ (eval-gate-demo, llm-finetune-demo, llm-routing-demo, security-scan-demo); samples/mlgear-titanic
Docs	docs/ (deploy-targets.md, secrets_design.md, mlgear-phase1-cost.md, demo-runbook.md, learning/task-types.md); ROADMAP.md
Packaging / editions	package.ps1, assemblies/, ee-platform/, ee-mlgear/, README.community.md

12. Sources (market section)

[1] Gartner press release, 9 Apr 2025 (opened): small task-specific models: https://www.gartner.com/en/newsroom/press-releases/2025-04-09-gartner-predicts-by-2027-organizations-will-use-small-task-specific-ai-models-three-times-more-than-general-purpose-large-language-models

[2] Epoch AI, "The plunging price of thought", 22 Sep 2026 (opened): https://epoch.ai/publications/the-plunging-price-of-thought

[3] Epoch AI, "Open models lag state-of-the-art closed models by 4 months", 29 May 2026 (opened): https://epoch.ai/data-insights/open-closed-eci-gap

[4] Belcak et al. (NVIDIA Research), "Small Language Models are the Future of Agentic AI" (excerpt): https://arxiv.org/abs/2506.02153

[5] Wang et al., "Dynamic LLM Routers are Often Misguided" (opened): https://arxiv.org/abs/2610.02762

[6] Futurum, "Enterprises Embrace Open Weights but Infrastructure Sellers Profit Most", Oct 2026 (excerpt, PDF not opened): https://futurumgroup.com/wp-content/uploads/2026/10/Enterprises-Embrace-Open-Weights-but-Infrastructure-Sellers-Profit-Most_AIONP202610.pdf

[7] Factory, "Why model routing must be in the harness" (excerpt, vendor claim): https://factory.com/news/model-routing-belongs-in-the-harness

[8] EU AI Act Service Desk, enforcement from 2 Aug 2026 (excerpt; verify): https://ai-act-service-desk.ec.europa.eu/en/ai-act/faq/commissions-enforcement-powers-related-ai-act-obligations-providers-most-advanced-models